

# CROSS-LAYER REPLACEMENT SHARING FOR MODEL COMPRESSION

Yen-Han Lin      Jen-Tzung Chien

Institute of Electrical and Computer Engineering,  
National Yang Ming Chiao Tung University, Hsinchu, Taiwan

## ABSTRACT

Cross-layer sharing provides an avenue to reduce large language model storage by reusing feed-forward weights across decoder depths, but effective sharing depends on which pretrained functions remain compatible when reused at different layers. We measure this compatibility through the directed full-model replacement cost and formulate the sharing under a fixed storage budget. A group replacement bound provides a practical criterion for grouping, while the final representative is selected in the actual donor-to-target direction. Across Llama-3.2-3B and Llama-3.1-8B, our method reaches 84.09% and 85.92% seven-task macro accuracy at the strongest structural operating points, compared with 65.06% and 69.96% for the strongest baseline. The advantage also persists after post-training quantization: the 25% 8B INT8 model occupies 6.50 GiB while remaining within 0.51 percentage points of the dense teacher. Structural analyses further explain the role of directionality and the gap between pairwise replacement and joint deployment.

**Index Terms**— large language model compression, cross-layer sharing, functional replacement, functional preservation, knowledge distillation

## 1. INTRODUCTION

Transformer decoders usually store an independent feed-forward network (FFN) at every layer. Cross-layer sharing reduces this redundancy by reusing a smaller set of FFN weights while retaining the original logical depth [1, 2]. Related compression methods share low-rank bases or approximate weight matrices through factorization [3, 4]. Their effectiveness depends not only on how many parameters are removed, but also on whether the resulting parameterization can recover task capability.

Weight distance and activation similarity provide only indirect evidence for cross-depth reuse. Two FFNs may appear locally similar yet cause different end-to-end changes when exchanged, and the effect can reverse with the replacement direction. We therefore score donor-to-target replacements inside the frozen pretrained model and use the resulting replacement cost to construct sharing groups.

Our method separates structural compression from subsequent recovery. We first select sharing groups and their representatives using only the frozen pretrained model, and then keep this structure fixed during all downstream training. This allows us to evaluate three distinct stages: the model immediately after compression, its ability to recover from ground-truth supervision, and its ability to recover with additional teacher information. We further test whether the recovered models remain effective after INT8 and INT4 post-training weight quantization.

Our contributions are threefold. First, we introduce a directed full-model replacement criterion that measures whether a pretrained FFN can serve another decoder depth. Second, we formulate sharing

under a fixed storage budget and derive a group replacement bound that leads naturally to complete-link grouping and directional representative selection. Third, we evaluate the resulting structures across two Llama backbones and three compression levels, including label-only recovery, teacher-assisted recovery, low-bit quantized deployment, and structural analyses that connect the measured replacement quantities to the final shared model.

## 2. RELATED WORK

**Cross-layer parameter sharing.** Parameter sharing reduces model size by reusing weights across network depth. Universal Transformers apply recurrent parameter reuse, while ALBERT shares parameters across encoder layers [1, 2]. More recently, Basis Sharing represents multiple LLM layers using shared low-rank bases with layer-specific coefficients [3]. These methods exploit redundancy across depth, but do not directly determine which pretrained functions can reliably serve other decoder depths. Our method addresses this assignment problem using directed full-model replacement cost rather than weight- or activation-level similarity.

**Low-rank model compression.** Low-rank factorization reduces storage by approximating weight matrices with smaller factors. SVD-LLM improves this process through truncation-aware data whitening and post-truncation parameter updates [4]. Unlike such layer-wise matrix factorization, our method reduces storage by sharing pretrained functions across decoder depths.

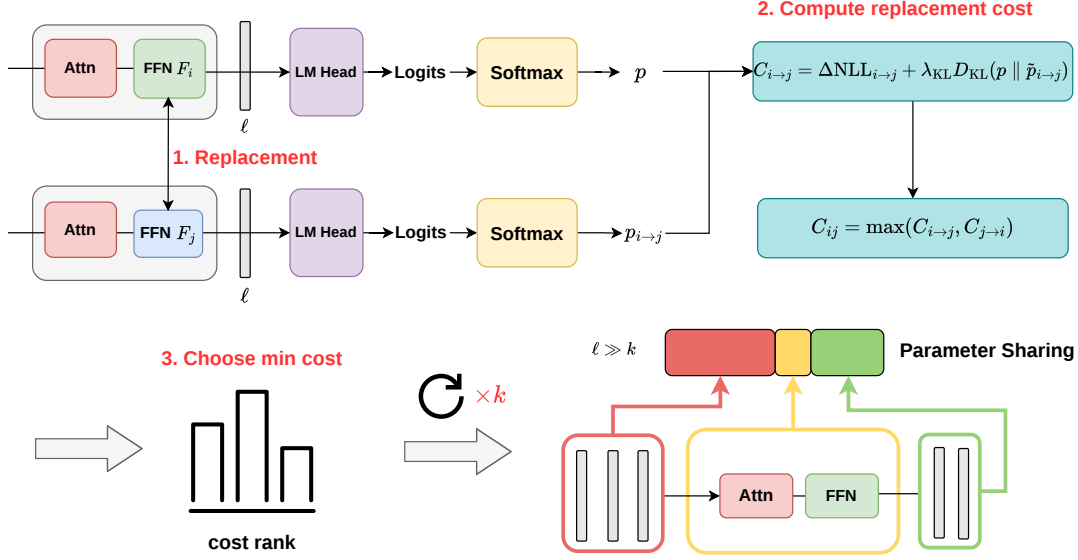
**Post-compression recovery.** Compressed models can recover task performance through supervised fine-tuning or knowledge distillation, where a teacher transfers information through its predictive distribution [5]. In our study, recovery is kept separate from structural selection and is used only to evaluate the recoverability of the frozen compression structure.

**Post-training quantization.** Post-training quantization further reduces model storage after structural compression. GPTQ uses approximate second-order information for weight quantization, while AWQ uses activation statistics to protect salient weights [6, 7]. We instead use a common training-free quantization procedure to test whether the structural advantage persists under INT8 and INT4 deployment.

## 3. BUDGETED FUNCTIONAL PRESERVATION

### 3.1. Directed replacement cost

Let  $M_0$  contain frozen decoder FFNs  $F_1, \dots, F_L$ . For donor  $i$  and target  $j \neq i$ , we replace  $F_j$  by  $F_i$  while keeping the remaining model fixed. Let  $p_0$  and  $p_{i \rightarrow j}$  denote the reference and intervened next-



**Fig. 1. Framework and evaluation logic.** Directed full-model replacements determine cross-depth compatibility, complete-link forms a fixed number of groups, and the selected representative initializes each shared FFN. The structure is then frozen and evaluated under Pure, CE, and CE+KD recovery.

token distributions. We define

$$C_{i \rightarrow j} = \underbrace{\text{NLL}(p_{i \rightarrow j}) - \text{NLL}(p_0)}_{\Delta \text{NLL}_{i \rightarrow j}} + \lambda_{\text{KL}} \mathbb{E}[D_{\text{KL}}(p_0 \| p_{i \rightarrow j})]. \quad (1)$$

The NLL term measures the observed-token change, while the KL term captures redistribution over the full predictive distribution, with  $\lambda_{\text{KL}} = 1$ . In general,  $C_{i \rightarrow j} \neq C_{j \rightarrow i}$ .

### 3.2. Budgeted replacement objective

A budget of  $K < L$  stored FFNs partitions the  $L$  layers into  $K$  sharing groups. For a sharing group  $\mathcal{G}$  and candidate representative  $m \in \mathcal{G}$ , define

$$\begin{aligned} d(m; \mathcal{G}) &= \max_{j \in \mathcal{G} \setminus \{m\}} C_{m \rightarrow j}, & d(m; \{m\}) &= 0, \\ \delta(\mathcal{G}) &= \min_{m \in \mathcal{G}} d(m; \mathcal{G}). \end{aligned} \quad (2)$$

Thus,  $\delta(\mathcal{G})$  is the best representative cost: the smallest worst-case replacement cost achievable by selecting one group member. Let  $\mathcal{P}$  denote a partition of the layer indices  $\{1, \dots, L\}$ . The ideal fixed-budget objective is

$$\mathcal{D}_K^* = \min_{\mathcal{P}: |\mathcal{P}|=K} \max_{\mathcal{G} \in \mathcal{P}} \delta(\mathcal{G}). \quad (3)$$

Thus,  $K$  fixes the number of stored FFNs, while  $\mathcal{D}_K^*$  is the minimum achievable worst-group cost under that budget.

### 3.3. Greedy replacement grouping

Joint optimization over partitions and representatives is combinatorial. We therefore define, for  $i \neq j$ , the mutual replacement cost and its corresponding group bound as

$$\bar{C}_{ij} = \max\{C_{i \rightarrow j}, C_{j \rightarrow i}\}, \quad \Delta(\mathcal{G}) = \max_{\substack{i, j \in \mathcal{G} \\ i \neq j}} \bar{C}_{ij}, \quad \Delta(\{i\}) = 0. \quad (4)$$

Here,  $\bar{C}_{ij}$  keeps the more costly replacement direction, while  $\Delta(\mathcal{G})$  is the largest mutual replacement cost within the group. For any representative  $m \in \mathcal{G}$  and  $j \in \mathcal{G} \setminus \{m\}$ ,  $C_{m \rightarrow j} \leq \bar{C}_{mj} \leq \Delta(\mathcal{G})$ ; hence

$$\delta(\mathcal{G}) \leq \Delta(\mathcal{G}). \quad (5)$$

Thus,  $\Delta(\mathcal{G})$  provides a conservative group replacement bound, while the final representative is selected using the directed replacement cost.

For two current groups, define

$$D_{\text{CL}}(A, B) = \max_{i \in A, j \in B} \bar{C}_{ij}. \quad (6)$$

The bound of the merged group satisfies

$$\Delta(A \cup B) = \max\{\Delta(A), \Delta(B), D_{\text{CL}}(A, B)\}. \quad (7)$$

At greedy step  $t$ , let  $\mathcal{P}_t$  denote the current partition and  $M_t = \max_{\mathcal{G} \in \mathcal{P}_t} \Delta(\mathcal{G})$  its maximum group bound. After merging  $A$  and  $B$ , the new maximum is  $\max\{M_t, D_{\text{CL}}(A, B)\}$ . Thus, choosing the candidate pair with minimum  $D_{\text{CL}}$  gives a one-step minimax complete-link rule without claiming global optimality.

After  $K$  groups remain, denote them by  $\mathcal{G}_1, \dots, \mathcal{G}_K$ . We then select each representative by

$$m_k^* = \arg \min_{m \in \mathcal{G}_k} \max_{j \in \mathcal{G}_k \setminus \{m\}} C_{m \rightarrow j}. \quad (8)$$

Consequently,

$$\delta_{\max} \equiv \max_k \delta(\mathcal{G}_k) \leq \max_k \Delta(\mathcal{G}_k) \equiv \Delta_{\max}. \quad (9)$$

In implementation, each logical layer  $\ell \in \mathcal{G}_k$  uses the FFN shared by group  $\mathcal{G}_k$  together with a layer-specific parallel low-rank adapter, following the low-rank residual parameterization of [8]. The effective FFN is

$$\tilde{F}_\ell(x) = F_{\text{shared}}^{(k)}(x) + A_\ell(x), \quad (10)$$

where  $x \in \mathbb{R}^D$  is the FFN input,  $F_{\text{shared}}^{(k)}$  is initialized from  $F_{m_k}^*$ , and  $A_\ell(x) = U_\ell V_\ell x$ , with  $V_\ell \in \mathbb{R}^{\rho \times D}$ ,  $U_\ell \in \mathbb{R}^{D \times \rho}$ , and  $\rho = 64$ . The shared FFN and adapter operate in parallel on the same layer input. All adapter parameters are included in the reported model size, and the sharing groups and representatives are fixed before recovery. Equation (9) characterizes isolated replacements; simultaneous sharing may introduce additional interactions, which we evaluate separately. To compare the simultaneous model with the single-replacement scale, let  $p_{\text{shared}}$  be the predictive distribution after deploying the complete sharing partition and define the per-layer joint cost

$$C^* = \frac{1}{L} [\text{NLL}(p_{\text{shared}}) - \text{NLL}(p_0) + \lambda_{\text{KL}} \mathbb{E}[D_{\text{KL}}(p_0 \| p_{\text{shared}})]] . \quad (11)$$

We use  $L = 28$  for Llama-3.2-3B and  $L = 32$  for Llama-3.1-8B.

## 4. EXPERIMENTS

### 4.1. Setup and evaluation

We evaluate Llama-3.2-3B and Llama-3.1-8B [9] at nominal compression targets of 15%, 20%, and 25%. We compare our method with Basis Sharing [3] and SVD-LLM [4]. Since the three methods represent compressed weights differently, the nominal targets are used to identify comparable operating points. Actual parameter reduction and serialized model size are measured separately.

Evaluation uses multiple-choice log-probability scoring on PIQA, Social IQa, WinoGrande, ARC-Challenge, ARC-Easy, HellaSwag, and OpenBookQA [10, 11, 12, 13, 14, 15]. We report the unweighted seven-task macro accuracy.

To separate structural compression from recovery, each frozen compressed checkpoint is evaluated under three settings. **Pure** uses no additional training, **CE** uses ground-truth supervision only, and **CE+KD** adds a frozen task-trained teacher for the corresponding backbone:

$$\mathcal{L}_{\text{CE+KD}} = \mathcal{L}_{\text{CE}} + \lambda_{\text{KD}} \tau^2 D_{\text{KL}}(p_T^{(\tau)} \| p_S^{(\tau)}) , \quad (12)$$

where  $p_T^{(\tau)}$  and  $p_S^{(\tau)}$  are the teacher and student predictive distributions at temperature  $\tau$ . CE and CE+KD use paired student seeds {42, 43, 44}. Training data, optimization budget, checkpoint selection, teacher, and evaluator are fixed across methods, and recovery never changes the compression structure selected beforehand.

**Recovery protocol.** Recovery uses 156,668/161,781 examples and 7,500/10,000 updates for 3B/8B, with maximum sequence length 384 and effective batch sizes 32/8. We use AdamW ( $\beta_1 = 0.9$ ,  $\beta_2 = 0.999$ , weight decay 0.01). Peak learning rates for the main model, shared FFNs, and layer-specific adapters are  $(3, 5, 8) \times 10^{-5}$  for 3B and  $(2.5, 4, 6) \times 10^{-5}$  for 8B, with warmup-cosine scheduling and gradient clipping at 1.0.

CE remains teacher-free, while CE+KD uses the same frozen teacher across methods and compression levels, with  $\tau = 2$  and  $\lambda_{\text{KD}} = 1$ . We validate every 500 updates on at most 2,048 held-out examples and retain the checkpoint with the lowest held-out decision CE; step 0 and final test sets are never used for checkpoint selection.

### 4.2. Recovery under increasing compression

Table 1 first shows that the compressed checkpoint alone is not a good measure of the final task model. Pure accuracy remains low for all three methods, although our method is slightly higher at several operating points. The main difference appears after recovery.

With CE+KD, our method achieves the highest macro accuracy at all six backbone-compression settings. At 15/20/25% compression, the advantage over the strongest baseline is 13.55/16.91/19.03 percentage points on 3B and 9.71/12.05/15.96 points on 8B. The gap becomes larger as compression becomes stronger.

From 15% to 25%, our CE+KD accuracy decreases by only 0.74 points on 3B and 0.22 points on 8B. In contrast, Basis Sharing decreases by 6.22/6.47 points and SVD-LLM by 7.20/5.67 points. Thus, the structure selected by our method remains much easier to recover as the storage budget becomes tighter.

CE-only recovery provides a second observation. On 3B, our method remains stable across all three compression levels. On 8B, CE becomes seed-sensitive at the harder operating points: the seed standard deviation decreases from 17.02 to 0.34 points at 20% and from 5.96 to 0.36 points at 25% when CE+KD is used. KD therefore mainly stabilizes difficult recovery settings rather than uniformly increasing the mean accuracy.

At 25% compression, our method also achieves the highest accuracy on all seven benchmarks for both backbones. Across 36 paired seed-level comparisons against Basis Sharing and SVD-LLM, all task-stratified bootstrap intervals remain above zero.

### 4.3. Structural validation

We next examine whether the structural measurements used by our method support the design choices in Sec. 3.

**Directional replacement.** Figure 2(a) shows clear donor-target asymmetry. The median  $|C_{i \rightarrow j} - C_{j \rightarrow i}|$  is 0.221 on 3B and 0.140 on 8B, while the 90th percentiles reach 7.31 and 6.24. This supports the directed cost  $C_{i \rightarrow j}$  rather than a symmetric similarity.

**Group replacement bound.** Figure 2(b) compares the group replacement bound  $\Delta(\mathcal{G})$  with the best representative cost  $\delta(\mathcal{G})$ . Every observed group satisfies  $\delta(\mathcal{G}) \leq \Delta(\mathcal{G})$ , as required by Eq. (5), with a maximum gap of 0.569. The gap measures how conservative the group bound is relative to the selected representative.

**Pairwise versus joint deployment.** Figure 2(c) compares the worst selected group cost with the per-layer simultaneous cost in Eq. (11). As compression increases,  $C^*$  rises from 0.619 to 1.193 on 3B and from 0.682 to 1.314 on 8B. The 20% and 25% structures can share the same  $\Delta_{\text{max}}$  while producing different  $C^*$ . All six observations lie above the dashed equality line, so pairwise quantities guide structural selection but do not numerically bound the simultaneous per-layer cost.

**Ablation.** At the 3B-20% operating point, the full method reaches 85.11% CE+KD accuracy with  $C^* = 1.099$ . Removing directionality lowers accuracy to 83.29% and raises  $C^*$  to 1.216. A symmetric donor has higher Pure accuracy (34.75%) but raises  $C^*$  to 3.132 and lowers CE+KD to 80.06%, showing that immediate Pure quality does not determine recoverability. Two-way mean and single-link yield the same frozen structure at this operating point and therefore are not independent comparisons.

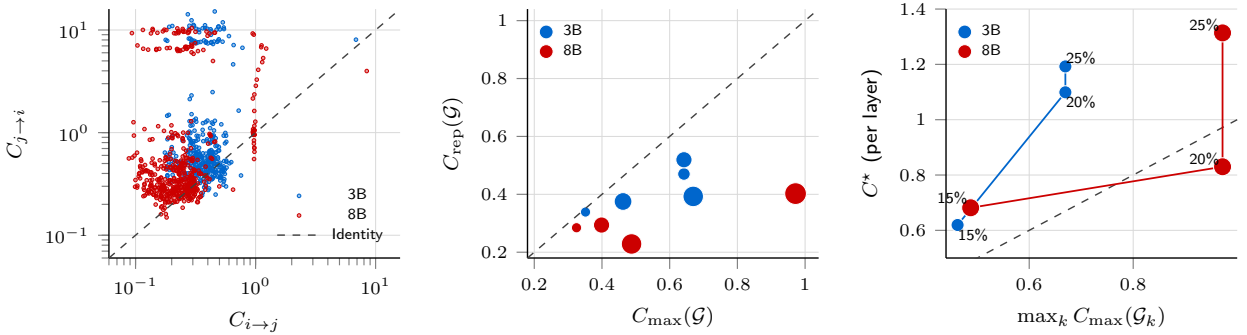
**Accounting and scope.** Basis Sharing and SVD-LLM closely match the nominal 15/20/25% standalone reductions, while ours realizes 13.95/20.93/25.58% on 3B and 15.26/19.63/26.17% on 8B, including all stored parameters. Logical depth and FFN execution count are unchanged, so we claim storage rather than inherent FLOP reduction.

### 4.4. Quantized deployment

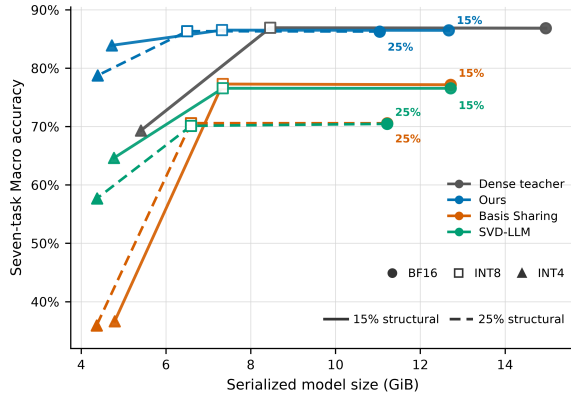
Finally, we test whether the structural advantage persists under low-precision deployment. We quantize the recovered Llama-3.1-

**Table 1. Seven-task macro accuracy (%).** Pure evaluates the frozen compressed checkpoint without training. CE and CE+KD report mean $\pm$ SD over paired seeds {42, 43, 44}. Bold denotes the best CE or CE+KD result at each operating point.

| (a) Llama-3.2-3B                                   |               |       |                                  |                                  | (b) Llama-3.1-8B                                   |               |       |                                  |                                  |
|----------------------------------------------------|---------------|-------|----------------------------------|----------------------------------|----------------------------------------------------|---------------|-------|----------------------------------|----------------------------------|
| Dense refs.: 36.93 pretrained / 82.30 task-trained |               |       |                                  |                                  | Dense refs.: 46.71 pretrained / 86.85 task-trained |               |       |                                  |                                  |
| Target                                             | Method        | Pure  | CE                               | CE+KD                            | Target                                             | Method        | Pure  | CE                               | CE+KD                            |
| 15%                                                | Basis Sharing | 33.20 | 71.04 $\pm$ 0.12                 | 71.28 $\pm$ 0.52                 | 15%                                                | Basis Sharing | 33.27 | 76.05 $\pm$ 0.39                 | 76.43 $\pm$ 0.47                 |
|                                                    | SVD-LLM       | 33.20 | 57.36 $\pm$ 20.32                | 70.78 $\pm$ 0.13                 |                                                    | SVD-LLM       | 33.21 | 74.43 $\pm$ 0.19                 | 75.31 $\pm$ 0.89                 |
|                                                    | Ours          | 33.21 | <b>85.35<math>\pm</math>0.17</b> | <b>84.83<math>\pm</math>0.47</b> |                                                    | Ours          | 36.27 | <b>85.54<math>\pm</math>0.33</b> | <b>86.14<math>\pm</math>0.33</b> |
| 20%                                                | Basis Sharing | 33.20 | 48.91 $\pm$ 13.84                | 68.15 $\pm$ 0.54                 | 20%                                                | Basis Sharing | 33.20 | <b>72.81<math>\pm</math>0.75</b> | 73.89 $\pm$ 0.54                 |
|                                                    | SVD-LLM       | 33.20 | 33.49 $\pm$ 0.22                 | 67.32 $\pm$ 0.28                 |                                                    | SVD-LLM       | 33.20 | 58.77 $\pm$ 21.74                | 73.01 $\pm$ 0.74                 |
|                                                    | Ours          | 33.95 | <b>85.07<math>\pm</math>0.54</b> | <b>85.06<math>\pm</math>0.08</b> |                                                    | Ours          | 33.65 | 68.04 $\pm$ 17.02                | <b>85.94<math>\pm</math>0.34</b> |
| 25%                                                | Basis Sharing | 33.22 | 33.73 $\pm$ 0.48                 | 65.06 $\pm$ 0.20                 | 25%                                                | Basis Sharing | 33.20 | 33.16 $\pm$ 0.04                 | 69.96 $\pm$ 0.64                 |
|                                                    | SVD-LLM       | 33.22 | 33.62 $\pm$ 0.37                 | 63.58 $\pm$ 0.73                 |                                                    | SVD-LLM       | 33.21 | 33.32 $\pm$ 0.26                 | 69.64 $\pm$ 0.34                 |
|                                                    | Ours          | 33.77 | <b>84.60<math>\pm</math>0.12</b> | <b>84.09<math>\pm</math>0.18</b> |                                                    | Ours          | 34.05 | <b>81.67<math>\pm</math>5.96</b> | <b>85.92<math>\pm</math>0.36</b> |



**Fig. 2. Structural validation.** (a) Directed replacement costs exhibit clear source–target asymmetry. (b) The best representation cost satisfies  $C_{\text{rep}}(\mathcal{G}) \leq C_{\text{max}}(\mathcal{G})$  for every observed group. (c) Per-layer joint cost  $C^*$  is compared directly with  $\max_k C_{\text{max}}(\mathcal{G}_k)$ ; the dashed line marks equality.



**Fig. 3. Quantized Llama-3.1-8B storage–accuracy trade-off.** BF16, INT8, and INT4 are shown at 15% and 25% structural compression; upper-left points are preferable.

8B models at the 15% and 25% structural operating points while keeping the sharing structure fixed.

**Quantization protocol.** We use training-free weight-only round-to-nearest quantization: INT8 is symmetric per output channel, INT4 is group-128, and activations remain BF16. The same layer policy and evaluator are used for all methods; shared FFNs and private adapters are both eligible. Packed weights are dequantized for BF16 compute,

so we make no integer-kernel speed claim. No calibration, QAT, or post-quantization recovery is used.

For ours, INT8 reaches 86.52% at 7.32 GiB for 15% structural compression and 86.34% at 6.50 GiB for 25%, reducing dense-BF16 storage by 51.07% and 56.53% while remaining 0.33 and 0.51 points below the teacher. The latter saves another 0.82 GiB over 15% INT8 for 0.18 points; small BF16–INT8 differences are not interpreted as gains.

Under INT4, ours loses 2.57/7.52 points at 15/25%, versus 40.51/34.63 for Basis Sharing and 11.93/12.77 for SVD-LLM. At 15%, ours still reaches 83.91%, compared with 36.65% and 64.62%, respectively.

**Conclusion.** Directed replacement yields recoverable sharing; 25% INT8 retains 86.34% at 6.50 GiB and stays robust.

## 5. REFERENCES

- [1] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Lukasz Kaiser, “Universal transformers,” in *International Conference on Learning Representations*, 2019.
- [2] Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut, “ALBERT: A lite BERT for self-supervised learning of language representations,” in *International Conference on Learning Representations*, 2020.
- [3] Jingcun Wang, Yu-Guang Chen, Ing-Chao Lin, Bing Li, and Grace Li Zhang, “Basis sharing: Cross-layer parameter sharing for large language model compression,” in *International Conference on Learning Representations*, 2025.
- [4] Xin Wang, Yu Zheng, Zhongwei Wan, and Mi Zhang, “SVD-LLM: Truncation-aware singular value decomposition for large language model compression,” in *International Conference on Learning Representations*, 2025.
- [5] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean, “Distilling the knowledge in a neural network,” in *NeurIPS Deep Learning and Representation Learning Workshop*, 2015.
- [6] Elias Frantar, Saleh Ashkboos, Torsten Hoeffer, and Dan Alistarh, “GPTQ: Accurate post-training quantization for generative pre-trained transformers,” in *International Conference on Learning Representations*, 2023.
- [7] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han, “AWQ: Activation-aware weight quantization for LLM compression and acceleration,” in *Proceedings of Machine Learning and Systems*, 2024, vol. 6.
- [8] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen, “LoRA: Low-rank adaptation of large language models,” in *International Conference on Learning Representations*, 2022.
- [9] Aaron Grattafiori et al., “The Llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024.
- [10] Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi, “PIQA: Reasoning about physical commonsense in natural language,” in *AAAI Conference on Artificial Intelligence*, 2020.
- [11] Maarten Sap, Hannah Rashkin, Derek Chen, Ronan Le Bras, and Yejin Choi, “Social IQA: Commonsense reasoning about social interactions,” in *Conference on Empirical Methods in Natural Language Processing and International Joint Conference on Natural Language Processing*, 2019.
- [12] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi, “WinoGrande: An adversarial winograd schema challenge at scale,” in *AAAI Conference on Artificial Intelligence*, 2021.
- [13] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord, “Think you have solved question answering? try ARC, the AI2 reasoning challenge,” in *arXiv preprint arXiv:1803.05457*, 2018.
- [14] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi, “HellaSwag: Can a machine really finish your sentence?,” in *Annual Meeting of the Association for Computational Linguistics*, 2019.
- [15] Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal, “Can a suit of armor conduct electricity? a new dataset for open book question answering,” in *Conference on Empirical Methods in Natural Language Processing*, 2018.